H2 heuristic of sequential entropy pooling

Introduction

The H2 heuristic is a practical approach within the sequential entropy pooling(SEP) framework for incorporating investor views into a prior distribution in a step-by-step manner. While the traditional EP method incorporates all investor views in a single step—often relying on heuristic fixes to handle nonlinear constraints—the H2 heuristic introduces a sequential strategy.
In H2 heuristics, the posterior distribution from one step becomes the prior for the next step. This contrasts with the H1 heuristics, which always uses the original prior p at each step, ignoring the updates made in earlier rounds.

Motivation

Investor views often follow a dependency hierarchy:

- **Means** can be incorporated directly,
- **Variances** require a fixed mean,
- **Skewness** depends on both mean and variance,
- **Kurtosis** depends on all lower moments.

Classical EP forces users to fix these dependencies in advance—often by assuming prior values—which may introduce unintended biases. The H2 heuristic resolves this by updating the posterior distribution incrementally and using it to inform the next set of views.

This makes H2:

- More logically consistent,
- Flexible for nonlinear or higher-order moment views,
- Slightly faster than H1 in practice.

Conceptual overview of H2 heuristics

The H2 heuristic operates by decomposing the full set of investor views into ordered subsets—each corresponding to a class of statistical moments (mean, variance, skewness, etc.). Rather than forcing all constraints into one

optimization problem, H2 incrementally incorporates them in sequence. This respects the natural dependencies between views and prevents logical inconsistencies.
Here is the algorithm for H2 heuristics

Algorithm 2 (H2)

for $i \in \{0, 1, ..., I\}$
    if $\mathcal{V}_i \neq \emptyset$, compute $q_i = EP\left(\mathcal{V}^i, \theta_{i-1}, q_{i-1}\right)$ and $\theta_i = f\left(R, q_i\right)$
    else $q_i = q_{i-1}$ and $\theta_i = \theta_{i-1}$
if $\bar{\mathcal{V}} \neq \emptyset$, compute $q = EP\left(\mathcal{V}, \theta_I, q_I\right)$
else $q = q_I$
return q

Explanation of Algorithm 2 (H2):

- The set of views is partitioned into V0,V1,...,Vi where each Vi contains views that require parameters from previous stages (e.g., variance requires the mean to be fixed).
- The algorithm initializes with the original prior q−1=p.
- At each stage ii, if Vi=∅, the EP method is applied using:
    - Prior qi−1,
    - Previously computed parameters θi−1 as fixed,
    - Views in Vi.
- The output posterior qi is then used to compute updated parameters θi, which will be required by the next class of views.
- If there are additional linear views V¯ that don't fall under the hierarchical structure, these are applied at the end using the final posterior qIqI.
- The final posterior qq thus reflects **all investor views**, integrated in a manner consistent with their dependency structure.

This algorithm ensures that each class of views builds logically upon the previous, avoiding the need to pre-specify fixed parameters (as in the classical EP method). It also reduces the chance of infeasible or contradictory constraints, especially when incorporating higher-order moments.

Implementation

We now explain the H2 procedure through the actual code implementation.

Step 1: Incorporating mean views(C0 class)

Applied a mean view on Corp IG, setting its expected return to 10% using entropy pooling. The resulting posterior matches this constraint while minimally diverging from the prior.

```
# C0 views
mean_row = R[:, 1][np.newaxis]
A0 = np.vstack((np.ones((1, S)), mean_row))
b0 = np.array([[1.], [0.1]])
q0 = ft.entropy_pooling(p, A0, b0)

ft.simulation_moments(R_df, q0)
```

Explanation:-

1. R[:, 1]: Extracts the simulated returns of asset 1 across all scenarios.
2. mean_row: Converts it to a row vector for matrix operations.
3. A0: Constructs the constraint matrix with:
   Row 1: Sum of probabilities equals 1
   Row 2: Expected return of asset 1 equals 10%
4. b0: Right-hand side of the constraints.
5. ft.entropy_pooling(p, A0, b0): Solves an entropy minimization problem to adjust the prior p into a posterior q0 , satisfying the constraints.
6. ft.simulation_moments(R_df, q0) :Evaluates the updated moments (mean, volatility, skewness, kurtosis) under q0 , verifying the impact of the mean view.

Output:-

After applying the mean view, we get the updated (posterior) statistical moments across all assets:

	Mean	Volatility	Skewness	Kurtosis
Gov & MBS	0.052059	0.035737	-0.266208	2.265610
Corp IG	0.100000	0.044309	-0.181936	2.128673
Corp HY	0.096963	0.054581	-0.187032	3.268894
EM Debt	0.163537	0.080470	-0.513780	2.418053
DM Equity	0.138907	0.143354	-0.298493	2.541248
EM Equity	0.236400	0.252120	0.413821	4.009514
Private Equity	0.309116	0.298377	-0.188268	2.031546
Infrastructure	0.173938	0.155932	-0.087321	1.786859
Real Estate	0.096609	0.083638	-0.573675	2.526741
Hedge Funds	0.126293	0.074791	-0.099078	4.009661

Step 2: Incorporating Volatility views(C1 class)

Added a volatility constraint on EM Equity (variance ≤ 0.04), using the posterior from Step 1 as the new prior. This ensures both views (mean + variance) are satisfied.

```
# C1 views
G = np.vstack((vol_rows[-1, :], skew_row, -kurt_row))
h = np.array([[0.2**2], [-0.75], [-3.5]])
means0 = q0.T @ R

G1 = np.vstack((np.ones((1, S)),vol_rows[-1, :]))
h1 = np.array([[1.],[0.2**2]])
q1 = ft.entropy_pooling(q0, A0, b0,G1,h1)  # H2: Use q0 instead of p

ft.simulation_moments(R_df, q1)
```

Explanation:

- `vol_rows[-1, :]` selects the variance constraint row for EM Equity.
- `np.ones((1, S))` ensures the total probability still sums to 1.
- h1 sets:
    1. the first constraint to 1 (probabilities sum to 1),
    2. the second to 0.04 (variance ≤ 0.04 for EM Equity).

Output:-

	Mean	Volatility	Skewness	Kurtosis
Gov & MBS	0.054774	0.035654	-0.371903	2.323417
Corp IG	0.100000	0.044837	-0.170081	2.082054
Corp HY	0.088799	0.049755	-0.112197	3.493298
EM Debt	0.155484	0.078428	-0.382654	2.199343
DM Equity	0.125618	0.136872	-0.336705	2.008845
EM Equity	0.168497	0.179375	-0.297939	2.611375
Private Equity	0.285589	0.293248	-0.161561	1.781404
Infrastructure	0.173096	0.157480	-0.074595	1.765371
Real Estate	0.089374	0.084538	-0.400003	2.281638
Hedge Funds	0.115472	0.066862	-0.501841	2.197843

Step 3: Incorporating Skewness Views (C2 Class)

Introduced a skewness constraint on DM Equity (skewness ≤ –0.75), building upon earlier views. The H2 heuristic helps preserve logical consistency in sequential updates.

```
# C2 views
G = np.vstack((vol_rows[-1, :], skew_row, -kurt_row))
h = np.array([[0.2**2], [-0.75], [-3.5]])
means0 = q1.T @ R  # Use updated parameters from q1
```

Defines a complete set of views including variance, skewness, and kurtosis (though not all are used here).

```
G2 = np.vstack((np.ones((1, S)),vol_rows[-1, :],skew_row))
h2 = np.array([[1.],[0.2**2],[-0.75]])
q2 = ft.entropy_pooling(q1, A0, b0,G2,h2)  # H2: Use q1 instead of p
ft.simulation_moments(R_df, q2)
```

Here, we apply a skewness constraint on DM Equity, requiring skewness ≤ –0.75, while preserving previously imposed views on Corp IG (mean) and EM Equity (volatility). This is done by applying Entropy Pooling on q1, the posterior from Step 2.

Explanation:-

- `np.ones((1, S))`: Total probability = 1
- `vol_rows[-1, :]`: Variance of EM Equity ≤ 0.04
- `skew_row`: Skewness of DM Equity ≤ –0.75

Output:-

We noticed that:

- Skewness of DM Equity has moved closer to –0.75, even if not exactly equal (because it's an inequality constraint),
- Other values shifted slightly to accommodate the new view while minimizing divergence from q1.

	Mean	Volatility	Skewness	Kurtosis
Gov & MBS	0.057860	0.032510	-0.302020	2.512401
Corp IG	0.100000	0.038539	-0.048611	2.475275
Corp HY	0.078716	0.048820	-0.088511	3.314295
EM Debt	0.142632	0.074625	-0.154740	2.378083
DM Equity	0.054061	0.170007	-0.324998	2.192497
EM Equity	0.090298	0.199738	-0.377252	2.435274
Private Equity	0.170860	0.327146	0.098553	1.791084
Infrastructure	0.148807	0.150632	0.132242	1.943268
Real Estate	0.070727	0.084268	-0.213496	2.490613
Hedge Funds	0.090357	0.077991	-0.329630	2.020381

Step 4: Incorporating Kurtosis Views (C3 Class)

Added a kurtosis constraint on DM Equity (kurtosis ≥ 3.5). Final posterior satisfies all views and is compared against the original prior using relative entropy and ENS.

```
# C3 views
G = np.vstack((vol_rows[-1, :], skew_row, -kurt_row))
h = np.array([[0.2**2], [-0.75], [-3.5]])
means0 = q2.T @ R  # Use updated parameters from q2

G3 = np.vstack((np.ones((1, S)),vol_rows[-1, :],skew_row,-kurt_row))
h3 = np.array([[1.],[0.2**2],[-0.75],[-3.5]])
q3 = ft.entropy_pooling(q2, A0, b0,G3,h3)  # H2: Use q2 instead of p
ft.simulation_moments(R_df, q3)

relative_entropy2 = q3.T @ (np.log(q3) - np.log(p))  # Still calculate RE against original prior
effective_number_scenarios2 = np.exp(-relative_entropy2)
print(f'RE = {np.round(relative_entropy2[0, 0], 3)}.')
print(f'Relative ENS = {np.round(effective_number_scenarios2[0, 0], 3)}.')
```

```
RE = 4.391.
Relative ENS = 0.012.
```

We now incorporate a kurtosis view on DM Equity, enforcing kurtosis ≥ 3.5, by applying Entropy Pooling on q2(posterior from Step 3). This completes the H2 sequence while preserving all previously applied views on mean, volatility, and skewness.

Explanation:-

- We negate the kurtosis constraint:

$$-\text{kurtosis} \leq -3.5 \Rightarrow \text{kurtosis} \geq 3.5$$

Output:

Relative Entropy (RE) = 4.391
→ Indicates how far your final posterior is from the original prior

Effective Number of Scenarios (ENS) = 0.012
→ Tells us that only ~1.2% of scenarios carry most of the weight now

Step 5: Visual Comparison of Posterior Distributions using KDE Plots

Here we visually compared how the scenario probabilities and resulting distributions change across-

- The **original prior** distribution (before any views),
- The **posterior using standard EP** (all views applied in one go),
- The **posterior using H1 heuristic**,
- The **posterior using H2 heuristic.**

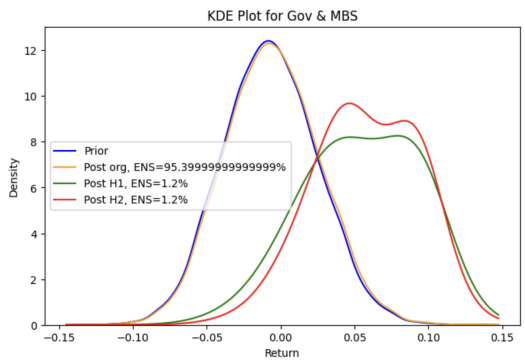
We generate Kernel Density Estimation (KDE) plots for each of the 10 asset classes, showing the effect of each method.

```
for i in range(10):
    plt.figure(figsize=(8, 5))
    # Prior distribution
    sns.kdeplot(x=R_df.iloc[:, i], label='Prior', color='blue')
    # Original EP posterior
    sns.kdeplot(x=R_df.iloc[:, i], weights=q[:, 0],
                label=f'Post org, ENS={100 * np.round(effective_number_scenarios[0, 0], 3)}%',
                color='orange')
    # H1 posterior
    sns.kdeplot(x=R_df.iloc[:, i], weights=q1[:, 0],
                label=f'Post H1, ENS={100 * np.round(effective_number_scenarios1[0, 0], 3)}%',
                color='green')
    # H2 posterior - NEW ADDITION
    sns.kdeplot(x=R_df.iloc[:, i], weights=q2[:, 0],
                label=f'Post H2, ENS={100 * np.round(effective_number_scenarios2[0, 0], 3)}%',
                color='red')

    plt.legend()
    plt.title(f'KDE Plot for {R_df.columns[i]}')
    plt.xlabel('Return')
    plt.ylabel('Density')
    plt.show()
```

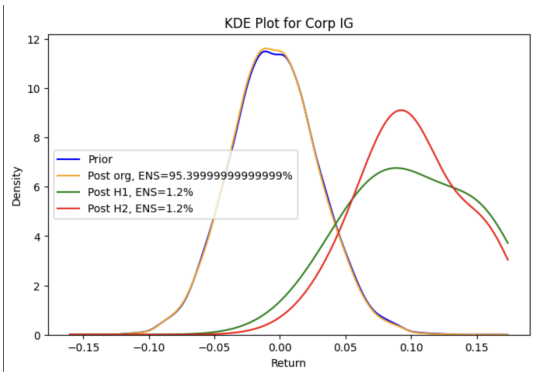
Here are the plots:-

1. KDE plot for Gov & MBS.



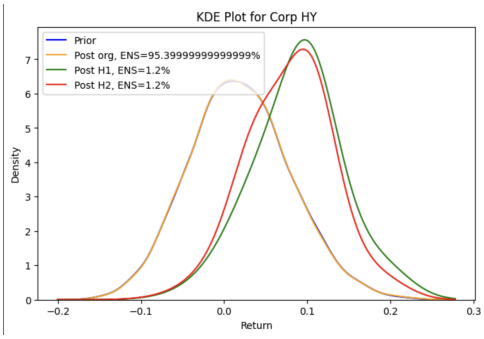
In this graph we saw,minor shift in the H2 posterior shows minimal impact, confirming that no direct views were applied on this asset.

2. KDE plot for Corp IG



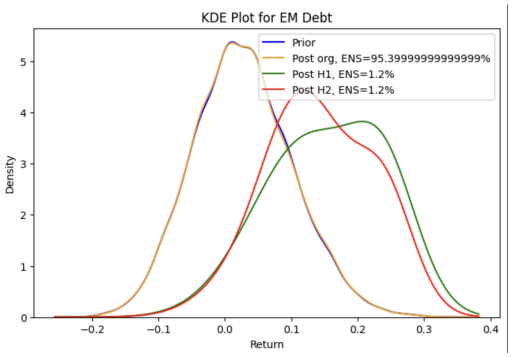
We can see that H2 posterior is clearly centered at the target 10% mean, as expected from the applied mean constraint.

3. KDE plot for Corp HY



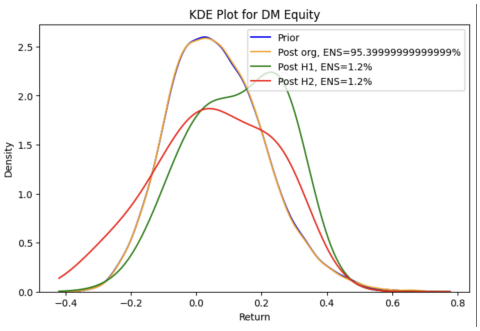
We can see that slight adjustments in the tail under H2 reflect indirect changes needed to accommodate views on correlated assets.

4. KDE plot for EM debt



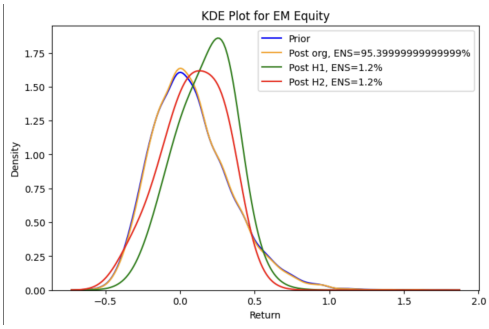
The plot reflects that distribution shape is slightly pulled to adapt to skew/kurtosis views on related risky assets like DM Equity.

5. KDE plot for DM Equity



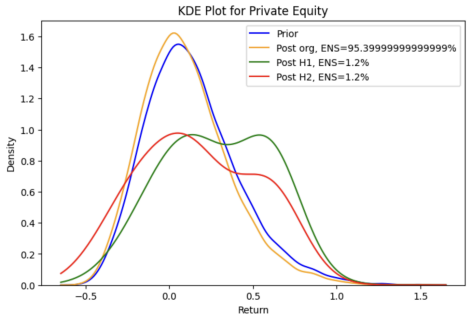
Noticeable left skew and fatter tails in the H2 posterior reflect the applied skewness (≤ –0.75) and kurtosis (≥ 3.5) views.

6. KDE plot for EM Equity



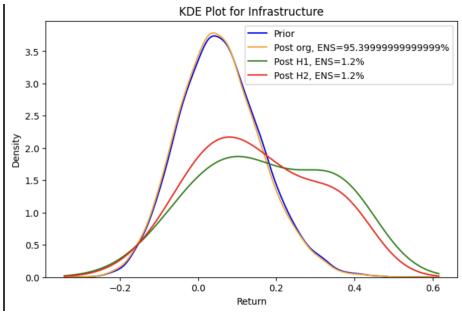
H2 distribution shows a tighter shape with reduced spread, satisfying the volatility constraint ($\sigma \leq 0.2$).

7. KDE plot for private equity



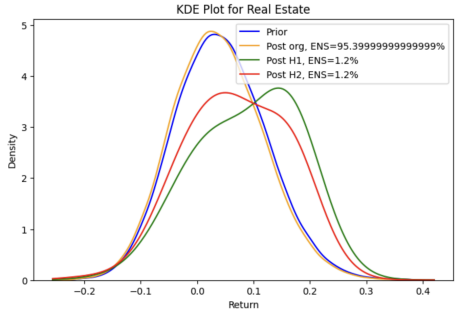
Minimal impact under H2 due to absence of direct views; differences mainly arise from global redistribution effects.

8. KDE plot for infrastructure



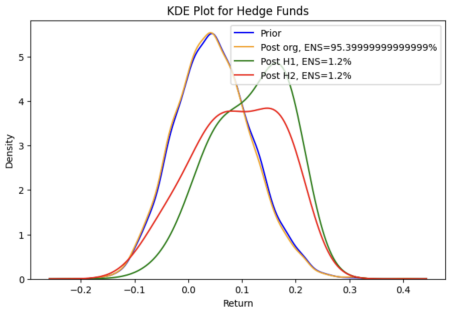
Distribution under H2 remains stable with slight smoothing, showing low sensitivity to views on other assets.

9. KDE plot for real estate



Minor tail adjustment in H2 due to redistribution of probabilities from skew/kurtosis-constrained assets.

10. KDE plot for Hedge funds



Slight right-shift in H2 posterior highlights how unconstrained assets can absorb residual adjustments from global views.

Conclusion:-

This implementation demonstrates how the H2 heuristic enables structured incorporation of interdependent views in a logically consistent manner. Results confirm that each class of views is satisfied while minimizing deviation from the prior, validating the practicality of Sequential Entropy Pooling.